Table des matières

What is Thread....2
What is Process....2
Multithreading and Multiprocessing....2
Multitasking(Multiprocessing and Multithreading)....2
Life cycle of a Thread (Thread States)....3
Implementation of Thread States....5
Java Threads | How to create a thread in Java....7
Thread Scheduler in Java....7
Thread Scheduler Algorithms....7
First Come First Serve Scheduling:....7
Time-slicing scheduling:....8
Preemptive-Priority Scheduling:....8
Thread.sleep() in Java....8
Can we start a thread twice....9
What if we call Java run() method directly instead start() method?....10
Java join() method....11
The Join() Method: InterruptedException....13
Priority of a Thread....13
Constants defined in Thread class:....13
Daemon Thread in Java....14
Java Thread Pool....14
Risks involved in Thread Pools....17
ThreadGroup in Java....17
Java Shutdown Hook....18
Synchronization in Java....18
Mutual Exclusive....18
Java Synchronized Method....19
Synchronized Block in Java....20
Static Synchronization....20
Why wait(), notify() and notifyAll() methods are defined in Object class not Thread class?....21
Interrupting a Thread:....22
Reentrant Monitor in Java....24

What is Thread

A **thread** in Java is a small unit of a program that runs independently to perform tasks simultaneously with other threads, helping the program do multiple things at once, Threads are **independent**, so it doesn't affect other threads

What is Process

A **process** in computing is an independent program that is being executed by the operating system. Each process has its own memory space and resources, which are isolated from other processes.

Multithreading and Multiprocessing

Multithreading in Java is a process of executing multiple threads simultaneously.

Multiprocessing and multithreading, both are used to achieve multitasking.

we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Java Multithreading is mostly used in games, animation, etc.

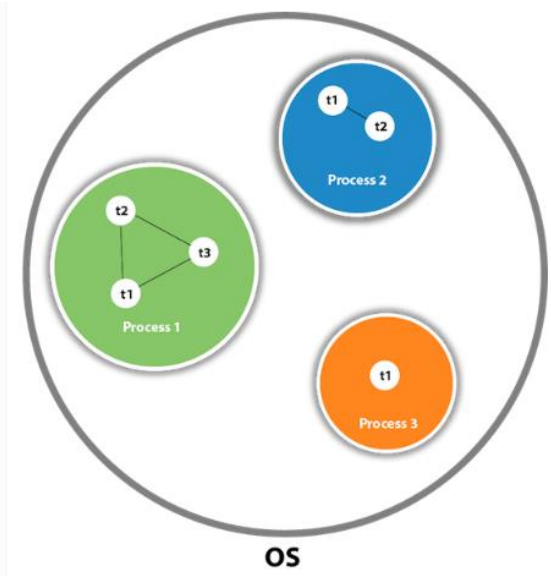
Multitasking(Multiprocessing and Multithreading)

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

Aspect	Process-based Multitasking (Multiprocessing)	Thread-based Multitasking (Multithreading)
Memory	Each process has its own address in memory (separate memory area).	Threads share the same address space (same memory area).
Weight	A process is heavyweight.	A thread is lightweight.
Communication Cost	Communication between processes is high because they have separate memory.	Communication between threads is low as they share the same memory space.
Creation and Management	Creating a new process requires more overhead and resources.	Creating and managing threads is more efficient and uses fewer resources.
Isolation	Processes are isolated; one process crashing does not affect others.	Threads are less isolated; a crash in one thread can affect the whole process.
Context Switching	Context switching between processes is slower and involves more overhead.	Context switching between threads is faster with less overhead.
Use Case	Best for running completely independent tasks (e.g., separate applications).	Best for running multiple tasks within the same application (e.g., parallel tasks).

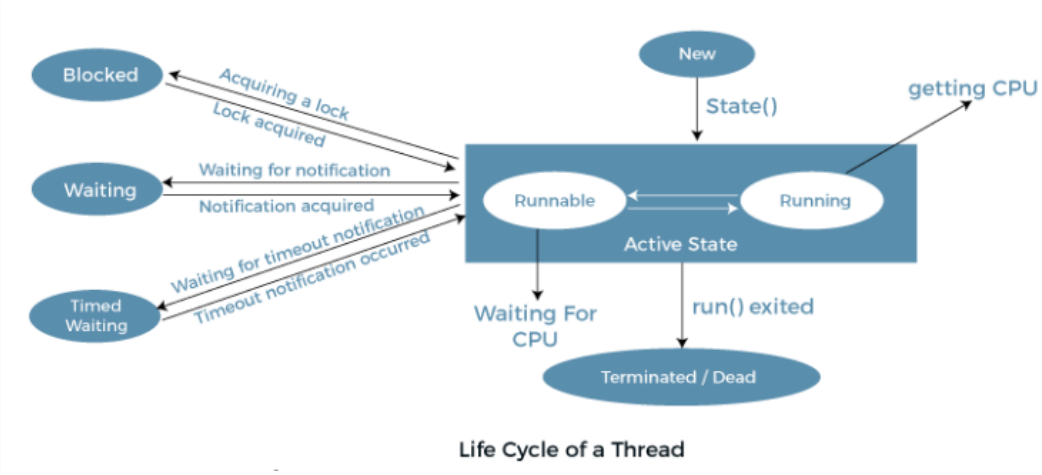
As shown in the above figure, a thread is executed inside the process. There is context-switching between the threads. There can be multiple processes inside the OS, and one process can have multiple threads.



Life cycle of a Thread (Thread States)

Thread always exists in any one of the following states. These states are:

1. New
2. Active
3. Blocked / Waiting
4. Timed Waiting
5. Terminated



Life Cycle of a Thread

In the **New** state, a thread is created but has not yet started execution. The code has not run, and the thread is waiting to be started.

In the **Active** state, a thread enters this state after invoking the start() method. It includes two sub-states:

- **Runnable**: The thread is ready to run but is waiting for CPU time.

1. In the runnable state, the thread may be running or may be ready to run at any given instant of time.
2. It is the duty of the thread scheduler to provide the thread time to run, i.e., moving the thread to the running state.

3. A program implementing multithreading assigns a fixed slice of time to each individual thread.
4. Each thread runs for a short span of time, and when that allocated time slice is over, the thread voluntarily gives up the CPU to another thread.
5. This allows other threads to run for their allocated time slice.
6. When such a scenario occurs, all threads that are ready to run, but waiting for their turn, remain in the runnable state.
7. In the runnable state, there is a queue where the threads wait for their turn to execute.

- **Running**: The thread is currently executing on the CPU. When the thread gets the CPU, it moves from the runnable to the running state. Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.

Blocked or Waiting: Whenever a thread is inactive for a period of time (not permanently), it is either in the blocked state or the waiting state.

- For example, a thread (let's say thread A) may want to print data from a printer. However, at the same time, another thread (thread B) is already using the printer.
- Therefore, thread A has to wait for thread B to finish using the printer. Thus, thread A is in the blocked state.
- A thread in the blocked state is unable to execute any instructions and does not consume any CPU cycles.
- Thread A remains idle until the thread scheduler reactivates it from the waiting or blocked state.
- When the main thread invokes the join() method, it enters the waiting state.
- The main thread waits for the child threads to complete their tasks.
- When the child threads finish, a notification is sent to the main thread, which moves it from the waiting state back to the active state.
- If many threads are in the waiting or blocked state, it is the thread scheduler's job to determine which thread to activate next.
- The chosen thread is then given the opportunity to run.

  A thread is in the **Blocked** state when it is waiting for a resource to become available.

  A thread is in the Waiting state when it is waiting for another thread to complete a task or notify it.

Timed Waiting: In some situations, a thread may have to wait for a long time to access a resource. This can lead to **starvation**, where a thread waits forever because it never gets a chance to run. To prevent this, a **timed waiting** state is used.

- Timed Waiting means the thread waits for only a specific period of time, not forever.
- After the waiting time is over, the thread automatically wakes up and can start running again.
- A real-world example of timed waiting is the use of the sleep() method. When a thread calls sleep(), it goes into the timed waiting state for a specified amount of time (e.g., 2 seconds). After the time runs out, the thread wakes up and resumes execution from where it left off.
- When a thread wakes up from the timed waiting state but doesn't immediately find a CPU available to run, it enters the Runnable state. In this state, the thread is ready to run but must wait for the thread scheduler to allocate CPU time.

Terminated: A thread reaches the termination state because of the following reasons:

- When a thread has finished its job, then it exists or terminates normally.
- Abnormal termination: It occurs when some unusual events such as an unhandled exception or segmentation fault.

- A terminated thread means the thread is no more in the system. In other words, the thread is dead, and there is no way one can respawn (active after kill) the dead thread.

Implementation of Thread States

//jawbo bla matchofo

```
The state of thread t1 after spawning it - NEW
The state of thread t1 after invoking the method start() on it - RUNNABLE
The state of thread t2 after spawning it - NEW
the state of thread t2 after calling the method start() on it - RUNNABLE
The state of thread t1 while it invoked the method join() on thread t2 -TIMED_WAITING
The state of thread t2 after invoking the method sleep() on it - TIMED_WAITING
The state of thread t2 when it has completed it's execution - TERMINATED
```

```
class Main {

    public static Thread t1;
    public static void main(String[] args) {
        System.out.println("JavaTpoint Java Compiler !!");

        // Creating an object of the class Main
        Main obj = new Main();
        t1 = new Thread(new ThreadState());

        // thread t1 is spawned
        // The thread t1 is currently in the NEW state.
        System.out.println("The state of thread t1 after spawning it - " + t1.getState());

        // invoking the start() method on
        // the thread t1
        t1.start();

        // thread t1 is moved to the Runnable state
        System.out.println("The state of thread t1 after invoking the method start() on it - " + t1.getState());
    }
}

class ABC implements Runnable {
    public void run() {
        try {
            // moving thread t2 to the state timed waiting
            Thread.sleep(100);
        } catch (InterruptedException ie) {
            ie.printStackTrace();
        }
        System.out.println("The state of thread t1 while it invoked the method join() on thread t2 -"
                + Main.t1.getState());
        try {
            Thread.sleep(200);
        } catch (InterruptedException ie) {
            ie.printStackTrace();
        }
    }
}

class ThreadState implements Runnable {
    public void run() {
        ABC myObj = new ABC();
        Thread t2 = new Thread(myObj);

        // thread t2 is created and is currently in the NEW state.
        System.out.println("The state of thread t2 after spawning it - " + t2.getState());
        t2.start();

        // thread t2 is moved to the runnable state
        System.out.println("The state of thread t2 after calling the method start() on it - " + t2.getState());

        // try-catch block for smooth execution
        try {
            // moving the thread t1 to the state timed waiting
            Thread.sleep(200);
        } catch (InterruptedException ie) {
            ie.printStackTrace();
        }
```

Java Threads | How to create a thread in Java

There are two ways to create a thread:

- By extending Thread class : Thread class provide constructors and methods to create and perform operations on a thread.
- By implementing Runnable interface : Runnable interface have only one method named run().

Extending Thread Class	Implementing Runnable Interface
The class that extends `Thread` cannot extend any other class because Java doesn't support multiple inheritance.	The class that implements `Runnable` can still extend another class, allowing more flexibility.
Easy to use when you don't need to extend another class.	Better suited when you need to extend another class, as it avoids the inheritance limitation.
A new instance of the `Thread` class is created directly.	The `Runnable` object is passed to the `Thread` constructor, allowing separation of the task from the thread.
Less commonly used as it is more restrictive.	More commonly used and preferred in most scenarios.

Thread Scheduler in Java

A component of Java that decides which thread to run or execute and which thread to wait.

There are two factors for scheduling a thread i.e. **Priority** and **Time of arrival**.

Priority: Priority of each thread lies between 1 to 10. If a thread has a higher priority, it means that thread has got a better chance of getting picked up by the thread scheduler.

Time of Arrival: Suppose two threads of the same priority enter the runnable state, then priority cannot be the factor to pick a thread from these two threads. In such a case, arrival time of thread is considered by the thread scheduler. A thread that arrived first gets the preference over the other threads.

Thread Scheduler Algorithms

First Come First Serve Scheduling:

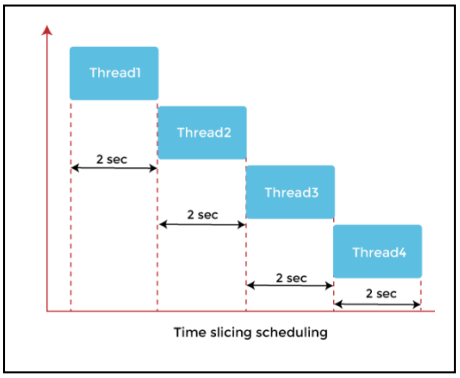
Threads	Time of Arrival
t1	0
t2	1
t3	2
t4	3

First Come First Serve Scheduling

- Starvation : Non-preemptive means that once a thread starts running, it keeps the CPU until it finishes. This can be problematic if a long-running thread is running, as other threads have to wait indefinitely. This is called starvation or infinite blocking, where some threads may never get a chance to execute if a long process is hogging the CPU.
- Convoy Effect : If a long task arrives first, it will block the shorter tasks from running. This is called the **convoy effect**, where many short tasks are forced to wait for one long task to finish, leading to poor performance and longer overall wait times.

- **No prioritization**: FCFS treats all processes equally, regardless of their priority or importance.

Time-slicing scheduling:



In the above diagram, each thread is given a time slice of 2 seconds. Thus, after 2 seconds, the first thread leaves the CPU, and the CPU is then captured by Thread2. The same process repeats for the other threads too.

- Choosing the right time slice (quantum) is tricky: If the time slice is too small, the CPU will spend more time on **context switching.**
- All tasks are treated equally, which means **critical tasks.**
- Time-slicing works best when tasks are similar in length ; If some tasks are very short and others are very long, time-slicing can lead to inefficiencies

Preemptive-Priority Scheduling:

Suppose there are multiple threads available in the runnable state. The thread scheduler picks that thread that has the highest priority. Since the algorithm is also preemptive, therefore, time slices are also provided to the threads to avoid starvation. Thus, after some time, even if the highest priority thread has not completed its job, it has to release the CPU because of preemption.

When two threads (Thread 2 and Thread 3) having the same priorities and arrival time, the scheduling will be decided on the basis of FCFS algorithm. Thus, the thread that arrives first gets the opportunity to execute first.

(There will always be some small difference in how the system handles the threads at a lower level, allowing FCFS to choose one to run first.)

Thread.sleep() in Java

```
1. public static void sleep(long mls) throws InterruptedException
2. public static void sleep(long mls, int n) throws InterruptedException
```

The method sleep() with the one parameter is the native method, and the implementation of the native method is accomplished in another programming language. The other methods having the two parameters are not the native method. That is, its implementation is accomplished in Java.

- mls: The time in milliseconds is represented by the parameter mls. The duration for which the thread will sleep is given by the method sleep().
- n: It shows the additional time up to which the programmer or developer wants the thread to be in the sleeping state. The range of n is from 0 to 999999. Developers might want a thread to sleep for a very precise amount of time. By specifying both milliseconds and nanoseconds .

When the Thread.sleep() method is called, it **pauses the execution of the current thread** for a specified amount of time. This means that the thread that invoked sleep() stops executing its code temporarily, and during this time, the CPU can run other threads or processes.

```
class TestSleepMethod1 extends Thread{
 public void run(){
  for(int i=1;i<5;i++){
  // the thread will sleep for the 500 milli seconds
    try{Thread.sleep(500);}catch(InterruptedException e){System.out.println(e);}
    System.out.println(i);
  }
 }
 public static void main(String args[]){
  TestSleepMethod1 t1=new TestSleepMethod1();
  TestSleepMethod1 t2=new TestSleepMethod1();

  t1.start();
  t2.start();
 }
}
```

```
1
1
2
2
3
3
4
4
```

Can we start a thread twice

No. After starting a thread, it can never be started again. If you does so, an IllegalThreadStateException is thrown. In such case, thread will run once but for second time, it will throw exception.

```
public class TestThreadTwice1 extends Thread{
 public void run(){
  System.out.println("running...");
 }
 public static void main(String args[]){
  TestThreadTwice1 t1=new TestThreadTwice1();
  t1.start();
  t1.start();
 }
}
```

```
running
Exception in thread "main" java.lang.IllegalThreadStateException
```

Remarque :

- When you write new Thread(), you are creating a thread object, but the thread does not start running yet. It is just prepared or ready to be started.
- To actually make the thread start running and do its job, you need to call the **start()** method.
- This is when the new thread is created and the code inside its run() method will start executing in that new thread
- Think of start() as creating a new worker that will help you do another task while the main worker (main thread) keeps working on something else.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Running in a new thread.");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();  // Creating the thread
        t.start();  // This starts a new thread and runs 'run()' in that new thread
        System.out.println("Main thread continues.");
    }
}
```

What if we call Java run() method directly instead start() method?

```
class TestCallRun1 extends Thread {
    public void run() {
        System.out.println("running...");
    }
}

public class Main {
    public static void main(String[] args) {
        TestCallRun1 t1 = new TestCallRun1();
        t1.run(); // This does NOT start a new thread, it
just calls the run() method like a normal method
    }
}
```

```
class TestCallRun1 extends Thread {
    public void run() {
        System.out.println("running...");
    }
}

public class Main {
    public static void main(String[] args) {
        TestCallRun1 t1 = new TestCallRun1();
        t1.start(); // This starts a new thread, which
runs the run() method in a separate call stack
    }
}
```

- **What happens:**
  - When you call `t1.run()`, it **doesn't** start a new thread.
  - Instead, it simply calls the `run()` method in the **current thread** (in this case, the main thread). This is just a normal method call, like calling any other method in your class.
- **Effect:**
  - There is **no parallel execution**. The `run()` method runs in the **main thread** and not in a new thread. It will print "**running...**" in the main thread's call stack.

- **What happens:**
  - When you call `t1.start()`, a **new thread** is created and the JVM starts running the `run()` method in a **separate thread** (a separate call stack).
  - The `start()` method takes care of creating a new thread for the `run()` method to execute concurrently with the main thread.
- **Effect:**
  - This will create a **new thread** that runs the `run()` method independently of the main thread. So the program has two threads: the **main thread** and the **new thread** created by `start()`.
  - The message "**running...**" is printed by the **new thread**.

there is no context-switching because here t1 and t2 will be treated as normal object not thread object.

```
class TestCallRun2 extends Thread{
 public void run(){
  for(int i=1;i<5;i++){
    try{Thread.sleep(500);}catch(InterruptedException e){System.out.println(e);}
    System.out.println(i);
  }
 }
 public static void main(String args[]){
  TestCallRun2 t1=new TestCallRun2();
  TestCallRun2 t2=new TestCallRun2();

  t1.run();
  t2.run();
 }
}
```

1
2
3
4
1
2
3
4

Java join() method

- The join() method in Java, which is part of the java.lang.Thread class, allows one thread to wait for another thread to complete its execution before continuing with the next line of code.
- Suppose you have a thread object th (which is an instance of Thread) and that thread is currently running.
- When you call th.join(), it makes the current thread (the one calling join()) wait until the thread th finishes its execution.
- In simple terms, join() ensures that a particular thread completes before the program proceeds to execute further instructions.

Hemm i like those examples :

Example1 :

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
            try {
                Thread.sleep(500);  // Thread sleeps for 500ms between each print
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();

        try {
            t1.join();  // Main thread waits for t1 to finish
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Thread t1 has finished, now continuing with main
thread.");
    }
}
```

```
1 2 3 4 5 Thread t1 has finished,
now continuing with main
thread.
```

Example2 :

```
class ThreadJoin extends Thread {
    // Overriding the run method
    public void run() {
        for (int j = 0; j < 2; j++) {
            try {
                // Sleeping the thread for 300 milliseconds
                Thread.sleep(300);
                System.out.println("The current thread name is: " + Thread.currentThread()
            }
            // Catch block for catching the raised exception
            catch (Exception e) {
                System.out.println("The exception has been caught: " + e);
            }
            System.out.println(j);
        }
    }
}
```

```
public class ThreadJoinExample {
   public static void main(String argvs[]) {

      ThreadJoin th1 = new ThreadJoin();
      ThreadJoin th2 = new ThreadJoin();
      ThreadJoin th3 = new ThreadJoin();

      // Thread th1 starts
      th1.start();
      try {
         System.out.println("The current thread name is: " + Thread.currentThread().getName())
         th1.join();
      }
      catch (Exception e) {
         System.out.println("The exception has been caught " + e);
      }
      // Thread th2 starts
      th2.start() ;
      try {
         System.out.println("The current thread name is: " + Thread.currentThread().getName());
         th2.join();
      }
      catch (Exception e) {
         System.out.println("The exception has been caught " + e);
      }

      // Thread th3 starts
      th3.start();
   }
}
```

The current thread name is: main
The current thread name is: Thread - 0
0
The current thread name is: Thread - 0
1
The current thread name is: main
The current thread name is: Thread - 1
0
The current thread name is: Thread - 1
1
The current thread name is: Thread - 2
0
The current thread name is: Thread - 2
1

The Join() Method: InterruptedException

In Java, interrupting a thread is a way to signal that you want to stop or "interrupt" a thread that is currently doing some work. This doesn't stop the thread immediately, but it tells the thread, "Hey, you might need to stop what you're doing or handle this interruption."

- Interrupting a thread can be useful when a thread is doing something that takes a long time, like waiting or sleeping, and you want to tell it to wake up or stop waiting.
- When a thread is **sleeping** (using Thread.sleep()) or **waiting** for something, and another thread **interrupts** it, the sleeping or waiting thread will throw an InterruptedException.

Priority of a Thread

- Chaque thread a une priorité.
- Les priorités sont représentées par un nombre entre 1 et 10.
- En général, le planificateur de threads (thread scheduler) planifie les threads en fonction de leur priorité (scheduling préemptif).
- Cependant, cela n'est pas garanti, car cela dépend de la spécification de la JVM et du type de scheduling qu'elle choisit.
- Un programmeur Java peut aussi assigner explicitement les priorités des threads dans un programme Java.

public final int getPriority(): The java.lang.Thread.getPriority() method returns the priority of the given thread.

public final void setPriority(int newPriority): The java.lang.Thread.setPriority() method updates or assign the priority of the thread to newPriority. The method throws IllegalArgumentException if the value newPriority goes out of the range, which is 1 (minimum) to 10 (maximum).

Constants defined in Thread class:

1. public static int MIN_PRIORITY 1
2. public static int NORM_PRIORITY 5
3. public static int MAX_PRIORITY 10

```java
class MyThread extends Thread {
    public MyThread(String name) {
        super(name);
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println(getName() + " is executing with priority " + getPriority());
        }
    }
}

public class ThreadPriorityExample {
    public static void main(String[] args) {
        // Create three threads with different names
        MyThread t1 = new MyThread("Thread 1");
        MyThread t2 = new MyThread("Thread 2");
        MyThread t3 = new MyThread("Thread 3");

        // Set different priorities
        t1.setPriority(Thread.MIN_PRIORITY); // Priority 1
        t2.setPriority(Thread.NORM_PRIORITY); // Priority 5 (default)
        t3.setPriority(Thread.MAX_PRIORITY); // Priority 10

        // Start all threads
        t1.start();
        t2.start();
        t3.start();
    }
}
```

```
Thread 3 is executing with priority 10
Thread 3 is executing with priority 10
Thread 3 is executing with priority 10
Thread 2 is executing with priority 5
Thread 2 is executing with priority 5
Thread 2 is executing with priority 5
Thread 1 is executing with priority 1
Thread 1 is executing with priority 1
Thread 1 is executing with priority 1
```

Daemon Thread in Java

- Daemon thread: A background thread that supports user threads.
- Automatically terminated by the JVM when all user threads complete.
- Examples: Garbage collection (gc) and finalizer.
- jconsole tool: Provides details about daemon threads, loaded classes, memory usage, and running threads.

Method	Description
public void setDaemon(boolean status)	is used to mark the current thread as daemon thread or user thread.
public boolean isDaemon()	is used to check that current is daemon.

User threads are regular threads created by the application to perform tasks. These threads are the primary threads in a Java program, and they keep the application running. The JVM continues to run as long as there is at least one user thread active.

In Java, if you want to convert a user thread into a daemon thread, you must call the setDaemon(true) method before starting the thread. If you attempt to set a thread as a daemon after it has been started, the JVM will throw an IllegalThreadStateException.

Java Thread Pool

- A Thread Pool is like a collection of worker threads that are ready to do tasks.
- Instead of creating new threads for every task, a fixed number of threads are created ahead of time.
- When a task comes in, a thread from the pool is assigned to do the work.
- After the thread finishes the task, it goes back to the pool and waits for the next task.

This approach makes it more efficient because threads are reused instead of being created and destroyed repeatedly.

Exemple1 :

```java
class WorkerThread implements Runnable {
    private String message;

    public WorkerThread(String s) {
        this.message = s;
    }

    public void run() {
        System.out.println(Thread.currentThread().getName() + " (Start) message = " + messag
        processmessage();
        System.out.println(Thread.currentThread().getName() + " (End)");
    }

    private void processmessage() {
        try {
            Thread.sleep(2000);  // Sleep for 2 seconds to simulate task processing
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```java
public class TestThreadPool {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(5); // Pool

        for (int i = 0; i < 10; i++) {
            Runnable worker = new WorkerThread("" + i);
            executor.execute(worker);  // Assigning the task to the thread
        }

        executor.shutdown();  // No new tasks will be accepted
        while (!executor.isTerminated()) {
            // Wait for all tasks to finish
        }

        System.out.println("Finished all threads");
    }
}
```

- Thread Pool optimizes resource usage by reusing a fixed number of threads.
- ExecutorService simplifies thread management.
- Tasks are queued and processed efficiently, even when more tasks exist than threads.
- Thread reuse helps avoid creating excessive threads and improves performance.
- Graceful shutdown ensures all tasks finish before the program exits.

```
pool-1-thread-1 (Start) message = 0
pool-1-thread-2 (Start) message = 1
pool-1-thread-3 (Start) message = 2
pool-1-thread-4 (Start) message = 3
pool-1-thread-5 (Start) message = 4
pool-1-thread-2 (End)
pool-1-thread-1 (End)
pool-1-thread-3 (End)
pool-1-thread-5 (End)
pool-1-thread-4 (End)
pool-1-thread-1 (Start) message = 5
pool-1-thread-3 (Start) message = 6
pool-1-thread-2 (Start) message = 7
pool-1-thread-4 (Start) message = 8
pool-1-thread-5 (Start) message = 9
pool-1-thread-2 (End)
pool-1-thread-1 (End)
pool-1-thread-3 (End)
pool-1-thread-5 (End)
pool-1-thread-4 (End)
Finished all threads
```

Exemple 2

```java
class Tasks implements Runnable {
    private String taskName;

    // Constructor of the class Tasks
    public Tasks(String str) {
        taskName = str;  // Initializing the field taskName
    }

    public void run() {
        try {
            for (int j = 0; j <= 5; j++) {
                Date dt = new Date();
                SimpleDateFormat sdf = new SimpleDateFormat("hh : mm : ss");

                if (j == 0) {
                    System.out.println("Initialization time for the task name: " + taskNam
                } else {
                    System.out.println("Time of execution for the task name: " + taskName
                }

                Thread.sleep(1000);  // Sleep for 1 second
            }
            System.out.println(taskName + " is complete.");
        } catch (InterruptedException ie) {
            ie.printStackTrace();
        }
    }
}
```

```java
public class Main {
    // Maximum number of threads in the thread pool
    static final int MAX_TH = 3;

    public static void main(String[] argvs) {
        // Creating five new tasks
        Runnable rb1 = new Tasks("task 1");
        Runnable rb2 = new Tasks("task 2");
        Runnable rb3 = new Tasks("task 3");
        Runnable rb4 = new Tasks("task 4");
        Runnable rb5 = new Tasks("task 5");

        // Creating a thread pool with MAX_TH number of threads (pool size is fixed)
        ExecutorService pl = Executors.newFixedThreadPool(MAX_TH);

        // Passes the Task objects to the pool to execute
        pl.execute(rb1);
        pl.execute(rb2);
        pl.execute(rb3);
        pl.execute(rb4);
        pl.execute(rb5);

        // Pool is shut down
        pl.shutdown();
    }
}
```

- You created a thread pool with a fixed size of 3 threads (MAX_TH = 3), meaning that only **3 threads** can run concurrently at any given time.
- Five tasks (task 1 to task 5) are submitted to the pool:
- Since the pool has a maximum of 3 threads, it can only run 3 tasks at the same time. Thus, task 1, task 2, and task 3 are picked up by the 3 available threads in the pool and start executing.
- The remaining tasks, task 4 and task 5, must wait in the queue until a thread becomes available.
- Once task 1, task 2, and task 3 complete their execution, the threads that were running those tasks become free.
- Now, the thread pool picks up task 4 and task 5 from the queue and starts executing them.

why first 3 tasks are done before executing of task 4 5 :
Initialization time for the task name: task 1 = 06 : 13 : 02
Initialization time for the task name: task 2 = 06 : 13 : 02
Initialization time for the task name: task 3 = 06 : 13 : 02
Time of execution for the task name: task 1 = 06 : 13 : 04
Time of execution for the task name: task 2 = 06 : 13 : 04
Time of execution for the task name: task 3 = 06 : 13 : 04
Time of execution for the task name: task 1 = 06 : 13 : 05
Time of execution for the task name: task 2 = 06 : 13 : 05
Time of execution for the task name: task 3 = 06 : 13 : 05
Time of execution for the task name: task 1 = 06 : 13 : 06
Time of execution for the task name: task 2 = 06 : 13 : 06
Time of execution for the task name: task 3 = 06 : 13 : 06
Time of execution for the task name: task 1 = 06 : 13 : 07
Time of execution for the task name: task 2 = 06 : 13 : 07
Time of execution for the task name: task 3 = 06 : 13 : 07
Time of execution for the task name: task 1 = 06 : 13 : 08
Time of execution for the task name: task 2 = 06 : 13 : 08
Time of execution for the task name: task 3 = 06 : 13 : 08
task 2 is complete. Initialization time for the task name: task 4 = 06 : 13 : 09 task 1 is complete. Initialization time for the task name: task 5 = 06 : 13 : 09 task 3 is complete.
Time of execution for the task name: task 4 = 06 : 13 : 10
Time of execution for the task name: task 5 = 06 : 13 : 10
Time of execution for the task name: task 4 = 06 : 13 : 11
Time of execution for the task name: task 5 = 06 : 13 : 11
Time of execution for the task name: task 4 = 06 : 13 : 12
Time of execution for the task name: task 5 = 06 : 13 : 12
Time of execution for the task name: task 4 = 06 : 13 : 13
Time of execution for the task name: task 5 = 06 : 13 : 13
Time of execution for the task name: task 4 = 06 : 13 : 14
Time of execution for the task name: task 5 = 06 : 13 : 14
task 4 is complete. task 5 is complete.

Risks involved in Thread Pools

- Deadlock:
  - This happens when all threads in the pool are busy waiting for **each other**.
  - For example, if every thread is waiting for results from other threads that are stuck in the queue (because there are no free threads to run them), the program gets stuck.
- **Thread Leakage**:
  - **Thread Leakage** occurs when a thread is taken from the thread pool to execute a task, but after completing (or during) the task, the thread **fails to return** to the pool for reuse..
  - This means the pool "loses" that thread, and over time, if too many threads are leaked, the pool may run out of threads, causing problems in handling tasks.(all the threads in the **thread pool** are either **busy** or have been **lost).**
- **Resource Thrashing:**
  - This happens when the thread pool is too large, causing too much time to be wasted on switching between threads.
  - If there are more threads than the system can handle, they might "starve" (not get enough resources), leading to poor performance.

ThreadGroup in Java

- ThreadGroup allows you to group multiple threads together into a single object.
- You can manage a group of threads as one, making it easier to suspend, resume, or interrupt them at once (though suspend(), resume(), and stop() are deprecated).
- Java's ThreadGroup is part of the java.lang.ThreadGroup class.
- A ThreadGroup can contain threads and other thread groups, forming a tree-like structure. Each thread group, except the root, has a parent group.
- A thread can access information about its own thread group, but it cannot access its parent thread group or other groups' information.

int activeGroupCount()

void destroy()

int enumerate()

int getMaxPriority()

ThreadGroup getParent()

boolean isDaemon()

```
public class ActiveCountExample {
    public static void main(String argvs[]) {
        ThreadGroup tg = new ThreadGroup("The parent group of threads");

        ThreadNew th1 = new ThreadNew("first", tg);
        System.out.println("Starting the first");

        ThreadNew th2 = new ThreadNew("second", tg);
        System.out.println("Starting the second");

        System.out.println("The total number of active threads are: " + tg.activeCount());
    }
}
```

```
class ThreadNew extends Thread {
    ThreadNew(String tName, ThreadGroup tgrp) {
        super(tgrp, tName);
        start();
    }

    public void run() {
        for (int j = 0; j < 1000; j++) {
            try {
                Thread.sleep(5);
            } catch (InterruptedException e) {
                System.out.println("The exception has been encountered " + e);
            }
        }
    }
```

Starting the first
Starting the second
The total number of active threads are: 2

Java Shutdown Hook

Purpose of the Shutdown Hook:

- The shutdown hook allows developers to execute code when the JVM is about to shut down.
- It is especially useful for performing cleanup tasks, like saving data or closing resources.

Why Use a Shutdown Hook?

- Normally, it's difficult to run code during unexpected shutdowns (e.g., kill signals or memory issues).
- The shutdown hook enables an arbitrary code block to run in these cases.

How to Implement a Shutdown Hook:

- Extend the Thread class and define cleanup code in the run() method.
- Register it with Runtime.getRuntime().addShutdownHook(new Thread()).
- To remove it, use Runtime.getRuntime().removeShutdownHook(Thread).

Common Use Cases:

- Closing files, logging out, sending alerts, or releasing resources when the JVM shuts down.

When JVM Shuts Down:

- When you press Ctrl+C in the command prompt.
- When System.exit(int) is called.
- During user logoff or system shutdown.

Synchronization in Java

Synchronization in Java is the capability to control the access of multiple threads to any shared resource.

There are two types of synchronization

1. Process Synchronization
2. Thread Synchronization

There are two types of thread synchronization mutual exclusive and inter-thread communication.

- Mutual Exclusive :
    - Synchronized method
    - Synchronized block.
    - Static synchronization.
- Cooperation (Inter-thread communication in java)

Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data. It can be achieved by using the following three ways:

1. By Using Synchronized Method
2. By Using Synchronized Block
3. By Using Static Synchronization

Java Synchronized Method

Concept of Lock in Java

Every object has a lock associated with it. By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them. EXplain acquire and release.

Synchronized method is used to lock an object for any shared resource.

When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

```
class Table {
    void printTable(int n) { // method not synchronized
        for(int i = 1; i <= 5; i++) {
            System.out.println(n * i);
            try {
                Thread.sleep(400);
            } catch(Exception e) {
                System.out.println(e);
            }
        }
    }
}
```

```
class MyThread1 extends Thread {
    Table t;

    MyThread1(Table t) {
        this.t = t;
    }

    public void run() {
        t.printTable(5);
    }
}

class MyThread2 extends Thread {
    Table t;

    MyThread2(Table t) {
        this.t = t;
    }

    public void run() {
        t.printTable(100);
    }
}
```

VS

```
class Table {
    synchronized void printTable(int n) { // synchronized metho
        for(int i = 1; i <= 5; i++) {
            System.out.println(n * i);
            try {
                Thread.sleep(400);
            } catch(Exception e) {
                System.out.println(e);
            }
        }
    }
}
```

problem without Synchronization

5

100

10

200

15

300

20

400

25

500

With Synchronization

5

10

15

20

25

100

200

300

400

500

```
class TestSynchronization1 {
    public static void main(String args[]) {
        Table obj = new Table(); // only one object shared by both threads
        MyThread1 t1 = new MyThread1(obj);
        MyThread2 t2 = new MyThread2(obj);

        t1.start();
        t2.start();
    }
}
```

Synchronized Block in Java

Suppose we have 50 lines of code in our method, but we want to synchronize only 5 lines, in such cases, we can use synchronized block.

If we put all the codes of the method in the synchronized block, it will work same as the synchronized method.

```
5
10
15
20
25
100
200
300
400
500
```

For same example :

```
class Table {
    void printTable(int n) {
        synchronized(this) { // synchronized block
            for(int i = 1; i <= 5; i++) {
                System.out.println(n * i);
                try {
                    Thread.sleep(400);
                } catch(Exception e) {
                    System.out.println(e);
                }
            }
        }
    } // end of the method
```

Static Synchronization

If you make any static method as synchronized, the lock will be on the class not on object.

```
class Table {
    static synchronized void printTable(int n) { // static synchronized meth
        for (int i = 1; i <= 5; i++) {
            System.out.println(n * i);
            try {
                Thread.sleep(400);
            } catch (Exception e) {
                System.out.println(e);
            }
        }
    }
}
```

- When printTable is static synchronized, it synchronizes at the class level (i.e., Table.class).
- This means only one thread can access the printTable method at any time, regardless of how many Table objects (instances) exist.
- Since printTable is static, it does not require an instance to be called. As a result, it ensures that if MyThread1 and MyThread2 both call Table.printTable, only one thread can execute printTable at a time, achieving class-level locking.

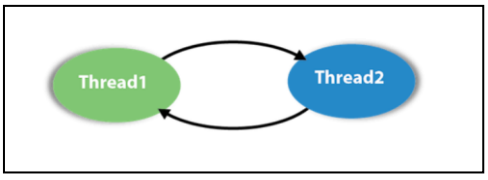
Synchronized block

```
static void printTable(int n) {
  synchronized (Table.class) {     // Synchronized block on class A
    // ...
  }
}
```

Deadlock in Java

Deadlock in Java is a part of multithreading.

Deadlock can occur in a situation when a thread is waiting for an object lock, that is acquired by another thread and second thread is waiting for an object lock that is acquired by first thread. Since, both threads are waiting for each other to release the lock, the condition is called deadlock.



```java
public class TestDeadlockExample1 {
    public static void main(String[] args) {
        final String resource1 = "ratan jaiswal";
        final String resource2 = "vimal jaiswal";

        // t1 tries to lock resource1 then resource2
        Thread t1 = new Thread() {
            public void run() {
                synchronized (resource1) {
                    System.out.println("Thread 1: locked resource 1");

                    try {
                        Thread.sleep(100);
                    } catch (Exception e) {}

                    synchronized (resource2) {
                        System.out.println("Thread 1: locked resource 2");
                    }
                }
            }
        };
```

```java
        Thread t2 = new Thread() {
            public void run() {
                synchronized (resource2) {
                    System.out.println("Thread 2: locked resource 2");

                    try {
                        Thread.sleep(100);
                    } catch (Exception e) {}

                    synchronized (resource1) {
                        System.out.println("Thread 2: locked resource 1");
                    }
                }
            }
        };

        t1.start();
        t2.start();
    }
}
```

Thread 1: locked resource 1
Thread 2: locked resource 2

A solution to deadlock lies in understanding how resources A and B are accessed. The key issue is the order in which these resources are locked. To resolve the deadlock, we just need to rearrange the code so that the shared resources are accessed in a consistent order.

Inter-thread Communication in Java

Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed.

- wait()
- notify() : The notify() method wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.
- notifyAll() : Wakes up all threads that are waiting on this object's monitor.

Method	Description
public final void wait()throws InterruptedException	It waits until object is notified.
public final void wait(long timeout)throws InterruptedException	It waits for the specified amount of time.

Why wait(), notify() and notifyAll() methods are defined in Object class not Thread class?

It is because they are related to lock and object has a lock.

wait vs sleep :

wait()	sleep()
The wait() method releases the lock.	The sleep() method doesn't release the lock.
It is a method of Object class	It is a method of Thread class
It is the non-static method	It is the static method
It should be notified by notify() or notifyAll() methods	After the specified amount of time, sleep is completed.

Example :

```
class Customer {
    int amount = 10000;

    synchronized void withdraw(int amount) {
        System.out.println("Attempting to withdraw...");

        if (this.amount < amount) {
            System.out.println("Insufficient balance; waiting for deposit...");
            try {
                wait();
            } catch (Exception e) {
                System.out.println("An error occurred: " + e.getMessage());
            }
        }
        this.amount -= amount;
        System.out.println("Withdrawal completed. Remaining balance: " + this.amount);
    }

    synchronized void deposit(int amount) {
        System.out.println("Depositing...");
        this.amount += amount;
        System.out.println("Deposit completed. New balance: " + this.amount);
        notify();
    }
}
```

```
class Test {
    public static void main(String args[]) {
        final Customer c = new Customer();

        // Thread for withdrawing
        new Thread() {
            public void run() {
                c.withdraw(15000);
            }
        }.start();

        // Thread for depositing
        new Thread() {
            public void run() {
                c.deposit(10000);
            }
        }.start();
    }
}
```

```
Attempting to withdraw...
Insufficient balance; waiting for deposit...
Depositing...
Deposit completed. New balance: 20000
Withdrawal completed. Remaining balance: 5000
```

Interrupting a Thread:

State of the Thread	Effect of `interrupt()`	Methods Related to Interruption
Sleeping or Waiting	- Exits sleeping or waiting state - Throws `InterruptedException`	- `interrupt()` - Causes an `InterruptedException` if sleeping/waiting
Active (not Sleeping/Waiting)	- Sets the interrupt flag to `true` - Does not immediately stop the thread	- `isInterrupted()` - Checks interrupt status without clearing it - `interrupted()` - Checks and clears interrupt status for the current thread

Example1 :

```
class TestInterruptingThread2 extends Thread {
    public void run() {
        try {
            Thread.sleep(1000);
            System.out.println("task");
        } catch (InterruptedException e) {
            System.out.println("Exception handled " + e);
        }
        System.out.println("thread is running...");
    }
```

```
    public static void main(String args[]) {
        TestInterruptingThread2 t1 = new TestInterruptingThread2();
        t1.start();

        t1.interrupt();
    }
```

Exception in thread "Thread-0" java.lang.RuntimeException: Thread interrupted...java.lang.InterruptedException: sleep interrupted

```
public static void main(String args[]) {
    TestInterruptingThread1 t1 = new TestInterruptingThread1();
    t1.start();
    try {
        t1.interrupt();
    } catch (Exception e) {
        System.out.println("Exception handled " + e);
    }
}
```

Exception handled

java.lang.InterruptedException: sleep interrupted

thread is running...

Exemple2 :

```
public class TestInterruptingThread4 extends Thread{

public void run(){
for(int i=1;i<=2;i++){
if(Thread.interrupted()){
System.out.println("code for interrupted thread");
}
else{
System.out.println("code for normal thread");
}

}//end of for loop
}

public static void main(String args[]){

TestInterruptingThread4 t1=new TestInterruptingThread4();
TestInterruptingThread4 t2=new TestInterruptingThread4();

t1.start();
t1.interrupt();

t2.start();

}
}
```

Code for interrupted thread
code for normal thread
code for normal thread
code for normal thread

Reentrant Monitor in Java

What is Reentrancy in Java?

- When a thread calls a synchronized method, it acquires a lock (monitor) on the object.
- With reentrant monitors, if that same thread tries to call another synchronized method on the same object, it doesn't need to reacquire the lock; it’s allowed to enter the method immediately.
- Without reentrancy, the thread would block, waiting for itself to release the lock, which would lead to a deadlock (the thread waiting on itself).

```
class Reentrant {
    public synchronized void m() {
        n();
        System.out.println("this is m() method");
    }
    public synchronized void n() {
        System.out.println("this is n() method");
    }
}
```